



The agent-based collaboration information system of product development

Wen-Yau Liang^{a,*}, Chun-Che Huang^b

^a Department of Information Management, Da Yeh University, 112 Shan-Jeau Rd, Fu Hsing, Ta Tsun, Changhua 510, Taiwan

^b Department of Information Management, National Chi-Nan University, Nantau, Taiwan

Abstract

Businesses are undergoing a major paradigm shift, moving from traditional management into a world of agile organizations and processes. An agile corporation should be able to rapidly respond to market changes. For this reason, corporations have been seeking to develop numerous information technology systems to assist with the information management of their business processes. Many of the on-coming new business processes may contain embedded agent-based information systems. Agent technology looks set to radically alter not only the way in which the computers interact, but also the way complex processes, e.g., product development, are conceptualized and built. This paper proposes an agent-based system of collaborative information and a solution procedure for designing with modules to develop modular products. This approach aims to address the research issue: “*How can modular product development be carried out through the intelligent agents of the collaboration information system to meet a customer’s requirements using modules that come from suppliers that are geographically separated and operate on differing computer platforms?*” © 2002 Elsevier Science Ltd. All rights reserved.

Keywords: Information systems; Agent technology; World wide web; Product development

1. Introduction

Business is undergoing a major paradigm shift, moving from traditional management into a world of agile organizations and processes. An agile corporation should be able to rapidly respond to market changes. Corporation managers make informed decisions based upon a combination of judgments and information from marketing, sales, research, development, manufacturing, and finance departments. Ideally, all relevant information should be brought

*Corresponding author. Tel.: +886-4-8511888 ext.3142.

E-mail addresses: wylang@mail.dyu.edu.tw (W.-Y. Liang), chuang@csie.ncnu.edu.tw (C.-C. Huang).

together before a judgment is exercised. However, obtaining pertinent, consistent and up-to-date information across a large company is a complex and time-consuming process. For this reason, corporations have been seeking to develop a number of information technology (IT) systems to assist with the information management of their business processes. Such systems aim to improve the way in which information is gathered, managed, distributed, and presented to people in key business functions and operations.

The notion of an intelligent agent (IA) is one of the most important concepts to emerge in IT systems in the 1990s, as Guilfoyle (1995) noted:

“In 10 years time most new IT development will be affected, and many business processes will contain embedded agent-based systems.”

Nwana (1996) views *intelligent agents* as “software components and/or hardware that are capable of acting exactly to accomplish tasks on behalf of its user and learn as they react and/or interact with their external environment”. The partial view of an agent typology is presented in Fig. 1. There are three main components: Cooperation, autonomy, and learning (Etzioni & Weld, 1995).

The potential of agent technology has been hailed in a 1994 report by Ovum (1994), a UK-based market research company, titled “Intelligent agents: the new evolution in software.” Some agent applications include personal information management, electronic commerce, business process management, message software, development tools, information management and retrieval, user interface, process control, workflow management, and network management (Jennings & Wooldridge, 1996). Agent technology looks set to radically alter not only the way in which computers interact, but also the way complex systems are conceptualized and built. The choice of agents as a solution technology was motivated by the following observations (Muller, Wooldridge, & Jennings, 1996):

- The domain involves an inherent distribution of data, problem-solving capabilities, and responsibilities.
- The integrity of the existing organizational structure and the autonomy of its sub-parts must be maintained.
- Interactions are fairly sophisticated, including negotiation, information sharing, and coordination.
- The problem solution cannot be prescribed entirely from start to finish.

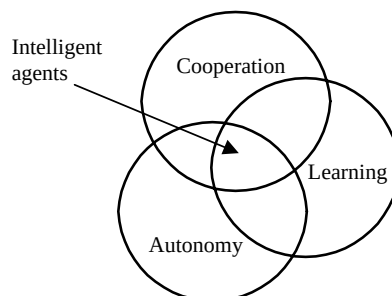


Fig. 1. A partial view of an agent typology.

When taken together, the following set of requirements leaves agents as the strongest solution candidate: (i) distributed object systems have the necessary encapsulation, but not the sophisticated reasoning required for social interaction or proactive behavior; and (ii) distributed processing systems deal with the distributed aspect of the domain but not with the autonomous nature of the components.

Product development is one of such business processes in which the sets of requirements must be satisfied. The need for agile industry to deliver high quality products at low cost has long been recognized. What is also becoming clear is that the further requirements of high variety and rapid product development are gradually being superimposed on these older requirements, so that, for example:

“The complex product markets of the twenty-first century will demand the ability to quickly and globally deliver a high variety of customized products.”

Earl Hall as quoted in Davidow and Malone (1992).

The problems involved in the product development are as follows:

- often of such size and complexity that no single individual, organization, or design environment is capable of effectively addressing all the aspects of the design,
- include “limited policies for recording experience” in acquiring and using system knowledge within companies, and
- “exhibit a disinclination to search for solutions” which is significant due to the scarcity of experts and support staff for information retrieval, etc., which relates to the knowledge access problem category.

One promising way for companies to address this difficult set of requirements is to use an *agent-based modular* approach, where products are built using standard modules by intelligent agents. This approach promises the benefits of high volume production (that arises from producing standard modules), while at the same time, being able to produce a high variety of products that are customized for individual customers through distributed systems. Such a modular product design starts to become a focus of attention and has been stated as a goal of good product development practice (Kidd, 1994).

A *modular product development* is one in which the input and output relationships between components, that is, the component interfaces, in a product have been fully specified and standardized. *Modular products* refer to products, assemblies and components that fulfill various functions through the combination of distinct modules (Pahl & Beitz, 1988). *Modules* refer to components whose functional, spatial, and other interface characteristics fall within the range of variations allowed by the specified standardized interfaces of a modular product development. The modular development of the products may lead to a large number of different products by creating distinct combinations of modules and components, thereby providing each product with distinctive functionality, features, and performance levels (Von Hippel, 1988; Sanchez & Ron, 1996). Modularity in a product development permits the processes of developing components for that design to be partitioned into tasks. Thus, modular product development can lead to an important form of *strategic flexibility* (Sanchez & Sudharshan, 1993), i.e., flexible product designs that allow a company to respond to changing markets and technologies by rapidly and

inexpensively creating product variants derived from different combinations of existing or new modules.

The potential benefits of modularity include the following (Nevins & Whitney, 1989; Pahl & Beitz, 1988):

- (a) *Economies of scale*. Since each module will usually be produced in relatively high quantities, natural economies of scale arise.
- (b) *Increased feasibility of product/component change*. Since each module interface is strictly specified, changes can be made to a module independent of other modules, provided the interfaces remain within specification.
- (c) *Increased product variety*. The use of modules mean that a high product variety can be achieved using different combinations of modules.
- (d) *Reduced order lead-time*. Since modules are manufactured in a relatively high volume, the logistics of production can be organized to reduce manufacturing lead time. Hence, the order lead time can be reduced.
- (e) *Decoupling tasks*. Since the interfaces and modules have been standardized, these interfaces enable design tasks to be decoupled and allow the production tasks to be decoupled. This decoupling can result in reduced task complexity and in the ability to complete tasks in parallel.
- (f) *The ease of product upgrade, maintenance, repair, and disposal*. Since a product is decomposed into modules, only certain modules need to be replaced when repair is made. For the same reason, upgrades, maintenance, and disposal are also made simpler.

Product variations based on mixing and matching modules now appear in product markets as diverse as aircraft, automobiles, consumer electronics, household appliances, personal computers (PCs), software, test instruments, and power tools (Morris & Ferguson, 1993; Sanchez, 1991).

For example, Swatch produces hundreds of different watch models, but can achieve this variety at a relatively low cost by assembling the variants from different combinations of standard modules (Pine II, 1992). A large number of different hands, faces, and wristbands can be combined with a relatively small selection of movements and cases to create seemingly endless combinations.

The Nippondenso panel meter is an example of designing with modules (Aoki, 1980). The example shows that old panel meter designs were redesigned to establish six standard kinds of modules. For example, the types of terminal components after the standardization have been reduced from 13 to four. A small variety of each is thus sufficient to support 288 different models, of which about 40 are currently being made.

The research issues associated with modular products can be divided into those associated with the *identification of modules*, the *design of modules*, and *designing with modules*. This paper is concerned with the latter area of designing with modules (DwM) that involves selecting the module combination to best satisfy the given set of requirements.

Important research issues in the area of designing with modules center around the central theme of selecting modules to satisfy customer requirements. The need for collaboration information to allow modular product development for environments where the development is carried out at

geographically separate locations, and on differing computer platforms, adds further complications to the research issues.

However, while modular product development appears to be an attractive proposition, little work has been done on these research issues (Pahl & Beitz, 1988; Ulrich & Tung, 1991), particularly for the relatively common development environment where various types of collaboration information are needed.

This paper aims to address the resulting research issue:

“How can designing with modules be carried out in an collaboration information system to meet customer requirements using modules that come from suppliers that may be geographically separated and where suppliers operate on differing computer platforms?”

The approach presented in this paper involves the use of the Internet and the world wide Web to aim at, what may be termed, remote collaboration for designing with modules. Some research has been done to facilitate remote collaboration. For example, Flexible Environment for Collaborative Software Engineering (FLECSE) is a multimedia environment designed to facilitate the communication of two or more geographically dispersed software engineers (Dewan & Riedl, 1993). Unfortunately, there is no work reported, as yet, that examines the collaboration with suppliers in general, or using the Internet in particular, or indeed the area of designing with modules.

This paper focuses on “remote collaboration” because of the geographical separation of module databases and servers, especially when geographically dispersed suppliers are involved. It proposes an agent-based system of collaboration information and the solution procedure for designing with modules to develop modular products. The implementation of the system for the collaborative product development through the world wide Web (WWW) is illustrated with an example of designing with modules for PCs.

2. Intelligent collaborative agent framework

In this section, an agent-based system called an intelligent collaborative agent (ICA) system is designed to enhance the existing product development process rather than to modify it, although the process may be modified before such a system is built. The ICA researches both the technology and the methods that are needed to improve the way information is gathered, managed, distributed and utilized by decision-makers in key business functions and operations. The system characteristics are as follows:

- **Intelligence:** The agent automatically customizes itself to the preferences of its customer (or client), based on previous experience and imprecise information from interaction with customers. The agent also automatically adapts to the changes in its environment.
- **Autonomous:** An agent is able to take the initiative and exercise a non-trivial degree of control over its own actions through service agreements.
- **Cooperation:** An agent does not blindly obey commands, but makes suggestions to modify requests or ask questions for clarification. It also cooperates with other agents to query the modules needed.

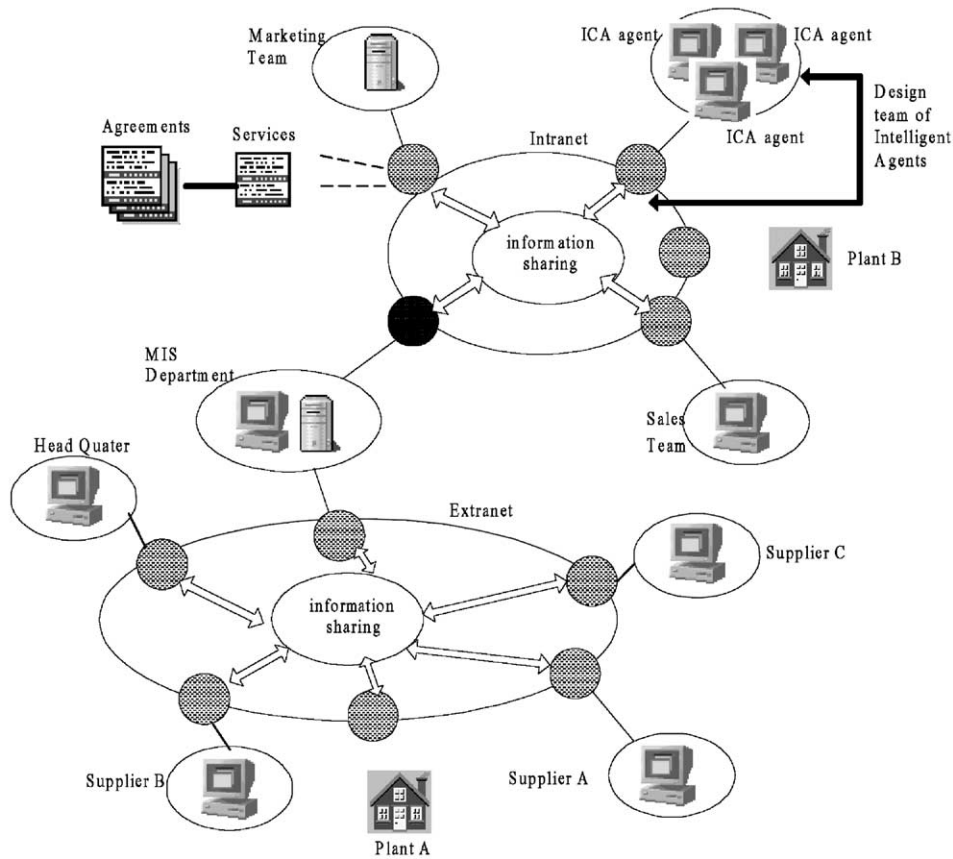


Fig. 2. The environment of the ICA system.

2.1. Intelligent collaborative agent system

In the ICA system, each ICA is able to perform one or more *services* (Fig. 2). A service corresponds to some problem-solving activities of module combination. The simplest service (called a *module combination job*) represents a problem-solving atomic activity endeavor in the ICA system, e.g., combining two modules into a higher level module. These atomic activities can be combined to form *complex services*, e.g., creation of a PC including a terminal, a motherboard, keyboard modules, etc., by adding ordering constraints (e.g., two tasks can run in parallel, must run in parallel, or must run in sequence). The nesting of services can be arbitrarily complex and at the topmost level the entire business process ultimately can be viewed as a service.

Service requirements are issued either from other departments, e.g., market teams through an Intranet, or from external customers through the Internet. Services are associated with one or more agents that are responsible for the management and execution of those services. Each service is managed by one agent, although the execution of its sub-services may involve a number of other agents. Since agents are autonomous, there are no control dependencies between them. Therefore, if an agent requires a service, which is managed by another agent, it cannot simply instruct that

agent to start the service. Rather, the agents must come to a mutually acceptable agreement about the terms and conditions under which the desired service will be performed. The mechanism for making agreements is *negotiation*—a joint decision-making process in which the parties verbalize their (possibly contradictory) demands and then move towards an agreement by a process of concession or search for new alternatives.

2.2. Architecture of an ICA

The activities of the agents involve the following:

- Selecting modules to satisfy the requirements of customers.
- Combining the modules into an integrated service.
- Coordinating and scheduling the processes intelligently.

All ICAs have the same basic architecture (Fig. 3). This involves an *agent body* that is responsible for managing the agent's activities and interacting with peers and an *agency* that represents the solution resources for the problems of product development processes. The body has a number of functional components responsible for each of its main activities—scheduling module combination operations, searching desired modules, optimizing module combination, and managing the module databases. This internal architecture is broadly based on the ADEPT (Jennings & Jennings, 1999), ARCHON (Jennings et al., 1996) agent models.

The domain resources include not only module databases and module combination jobs, but also other ICAs. The latter case allows a *nested (hierarchical) agent system* to be constructed in which higher level agents realize their functionality through lower level agents (the lower-level agents have the same structure as the higher level agents and, can, therefore, have sub-agents as well as module combination jobs in their agency). For example, the higher-level agent may represent a product development department whose work is carried out by a number of design teams (the lower-level agents). This structure enables flat, hierarchical, and hybrid organizations to be modeled in a single framework.

The ICA plays four roles in the design processes:

A *scheduler* is an agent who schedules the module combination operations and is responsible for assessing and monitoring the agent's ability to meet: (i) The customer agreement that is already agreed upon and (ii) the potential customer agreement that it may agree to in the future. This involves two main roles: scheduling and exception handling. The former involves maintaining a record of the availability of the agent's resources, which can then be used to determine whether customer agreement can be met or new customer agreement can be accepted. The exception handler receives exception reports from the optimizer during service execution (e.g., “service may fail”, “service has failed”, or “no customer agreement in place”) and decides upon the appropriate response. For example, if a service is delayed, then the scheduler may decide to locally reschedule it, to renegotiate its customer agreement, or to terminate it altogether.

An *optimizer* is an agent that optimizes the module combination based upon the requirements from customers and engineering constraints (Kusiak & Huang, 1997). Three main roles involve: service execution management—optimizing executed services as specified by the agent's customer

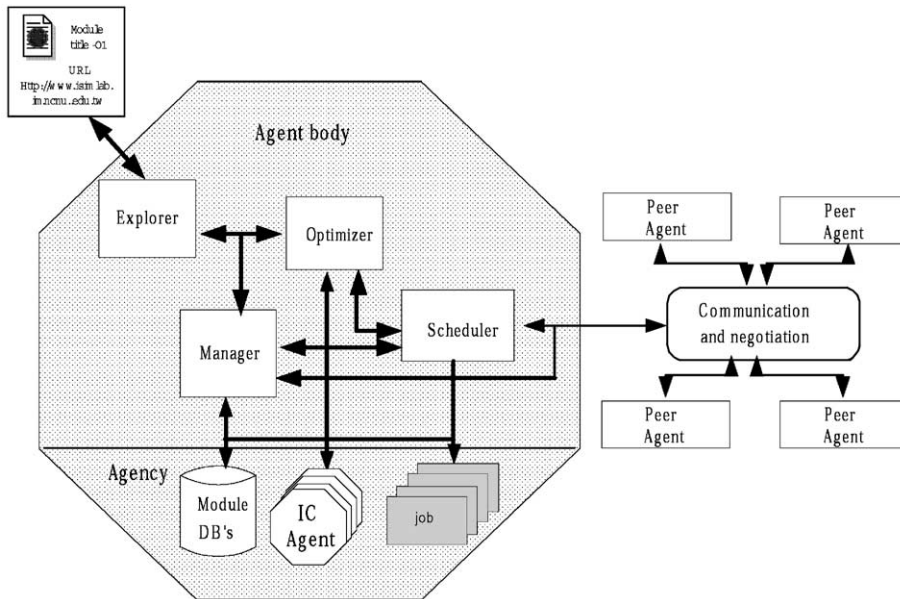


Fig. 3. The ICA architecture (adopted from Jennings et al., 1996).

agreements, solution presentation—routing solutions between servers, clients and other agents, and exception handling—monitor the execution of jobs and services for unexpected events and then react appropriately.

A *manager* is an agent who

- (i) maintains the module database where module information is stored,
- (ii) delivers the status messages of active services between optimizer and the clients, between an agent and its agency, and between peer agents, and
- (iii) communicates
 - between the optimizer and clients within the agency relating to job management activities (e.g., activate, suspend, or resume a job), and
 - between agents within that agency or peer agents relating to service execution management, e.g., an instruction to start service, service completion, and service results.

An *explorer* is an agent who searches the modules that are located in other distributed databases and performs the role of managing, querying or collating module information from many distributed sources. It is able to traverse the WWW, gather information and report what it retrieves to a home location. Fig. 4 depicts how the typical static explorer works.

3. Designing with modules procedure

Product development is often defined as the creation of synthesized solutions in the form of products, processes or systems that satisfy perceived needs through mapping between functional

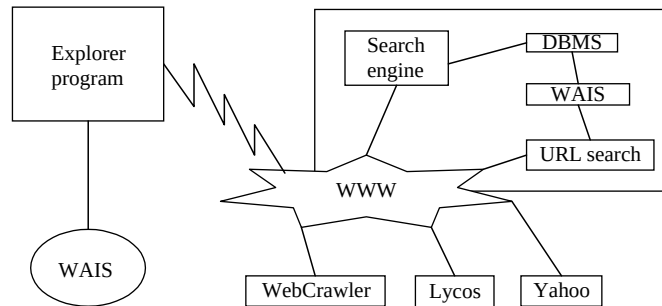


Fig. 4. A view of how an explorer works.

requirements (FRs) in the functional domain and the design parameters (DPs) in the physical domain, through the proper selection of DPs that satisfy FRs (Suh, 1990). The relationship between requirements and functions is discussed in Kota and Ward (1990), and Kusiak and Szczerbicki (1992).

In theory, it is possible to decompose functions so that the lowest level of the function structure consists exclusively of functions that cannot be sub-divided further, while remaining generally applicable. The relationship between sub-functions and overall function is often governed by certain constraints, in as much as some sub-functions have to be satisfied before others. The meaningful and compatible combination of sub-functions into overall function produces a so-called *functional structure* (Pahl & Beitz, 1988).

The functional structure may be represented in the form of networks where the blocks correspond to the sub-functions and the flow connects the blocks (see Fig. 5). The functional structure can be decomposed into multiple levels to separate the overall function into multiple sub-functions. The breakdown of functions to sub-functions provides the first step in the process of designing with modules.

The term functional requirement or requirement is used to denote the functionality of the design specifications which should be satisfied by the modules. Another distinction to be clarified is functional requirements and constraints. They are similar and facilitate the design activity and guide the search through the design solution space. However, while functional requirements concern the properties of the product to be designed, constraints are, in essence, related to the limitations or specifications of the module interfaces.

The following procedure with the four steps that allows the functional requirements to be satisfied by the modular design is presented. Note that some simplifications have been made to reduce the space required for the description. The details are described in (Huang & Liang, 2000):

- Refine the initial design specification into an equivalent set of functional requirements such that each requirement in the set is ready to be mapped into a set of modules.
- Build a null module model and initialize the design process.
- Map each functional requirement into a set of modules and add the desired module to the module model. Repeat this process until all functional requirements are mapped.
- Initiate all the remaining attributes and consider the relations between the modules.

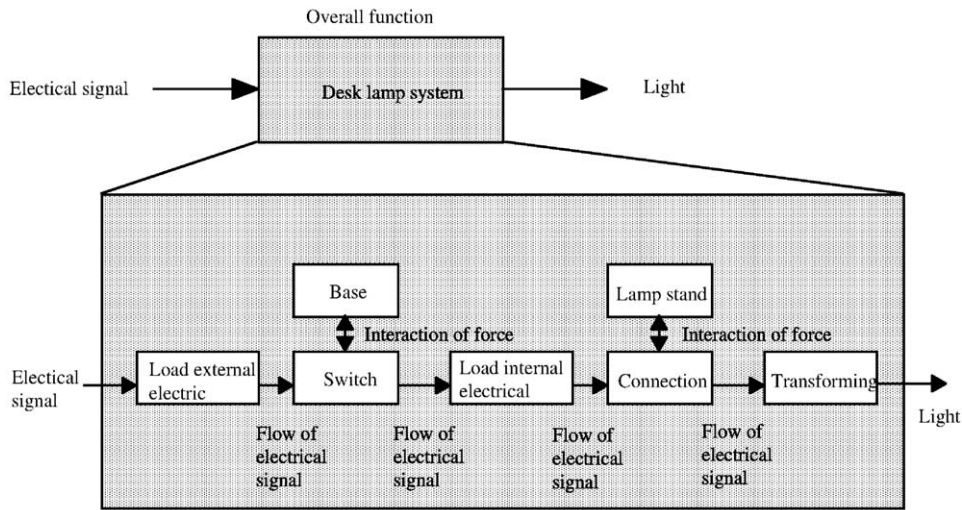


Fig. 5. Example of functional structure: desk lamp.

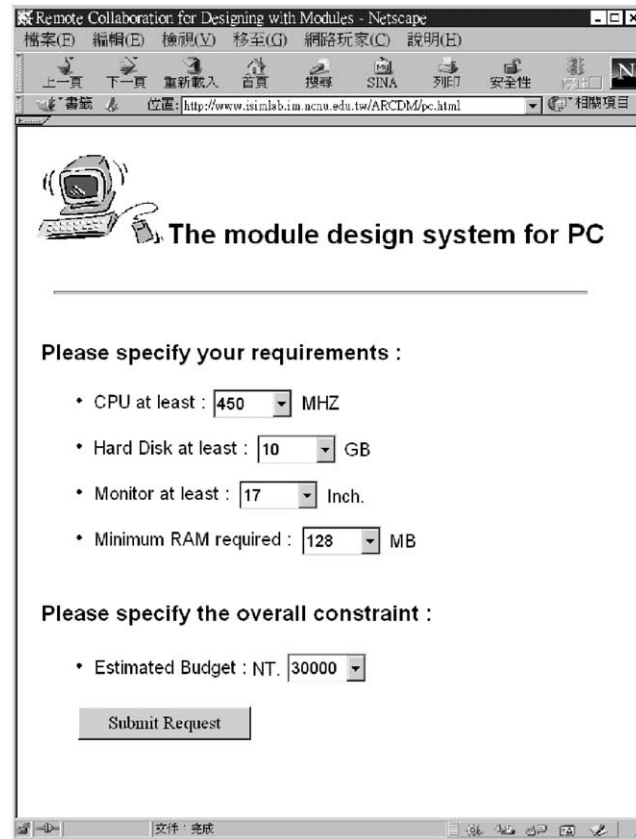
3.1. Example operation—personal computer design

The previous DwM procedure is implemented using the WWW to result in a prototype system. Decision makers or clients use the browsers to input the requirements through the WWW regardless of what platforms are used. The implementation domain shown is that of the design of PCs. Note that the example has been abridged from a full description, due to space limitations, and certain simplifications have been made to ease the description.

Compared to other products (e.g., a supercomputer or a minicomputer), a PC typically has a simpler operating system, a smaller amount of memory and primary storage space, limited input and output devices, and is a single-user system (Fohn, Liau, Greef, Young, & O'Grady, 1995). The explosion of software products and their inherent incompatibility problems and platform-specific requirements has added to the complexity of PC configuration. PCs are essentially modular products with standardized interfaces.

3.2. Example scenario

A scenario is considered where the PC modular products are designed. The PC configuration includes motherboard with CPU mounted on, e.g., Pentium II 450, and other modules such as RAM, monitor, and hard disk. This example presents the characteristics of the remote collaborative DwM, rather than focus on illustrating the entire and the detailed computer configuration. Numerous details, e.g., the other module, e.g., power supply module, are neglected in this paper in order to make this paper compact. The overall constraint is illustrated with the factor, budget: NT \$30,000. The I/O constraints, e.g., the number of RAM pins to plug in is checked by the module operator during the search of feasible solutions. The function evaluation and ranking is based on the function—cost, i.e., the feasible one with minimum cost ranked the



Remote Collaboration for Designing with Modules - Netscape

檔案(E) 編輯(E) 檢視(V) 移至(G) 網路玩家(C) 說明(H)

上一頁 下一頁 重新載入 首頁 搜尋 SINA 列印 安全性

書籤 位置: <http://www.isimlab.im.acnu.edu.tw/ARCDM/pc.html> 相關項目

 **The module design system for PC**

Please specify your requirements :

- CPU at least : MHZ
- Hard Disk at least : GB
- Monitor at least : Inch.
- Minimum RAM required : MB

Please specify the overall constraint :

- Estimated Budget : NT.

文件 完成

Fig. 6. Customers' requirements.

first. The function name is also represented with the common term, e.g., Pentium II 450. The concurrent engineering constraints in this example refer to the aggregated reliability of the modules assembled together and is determined as 0.976 arbitrarily.

3.3. Solution procedure

The initial design specifications are refined into functional requirements and this step may require user intervention. Customers or clients, e.g., the design team, use the browser to choose the service form, input the requirements, and send the form to the server, an ICA. The customer requirement through the WWW is illustrated in Fig. 6.

After the ICA receives the service form, the ICA package in the *agent body* is initiated. The *scheduler* assigns corresponding *explorers* to retrieve the required module information from the desired distributed databases. The *manager* then coordinates and monitors the processes between server/client, and informs the *optimizer* to run the DwM procedure when the transformed module information is ready. Finally, the results are presented to the client (Fig. 7).

The procedure searches solutions based on depth-first method. The depth-first method may obtain a final feasible solution quicker than the width-first method in the case where the solution

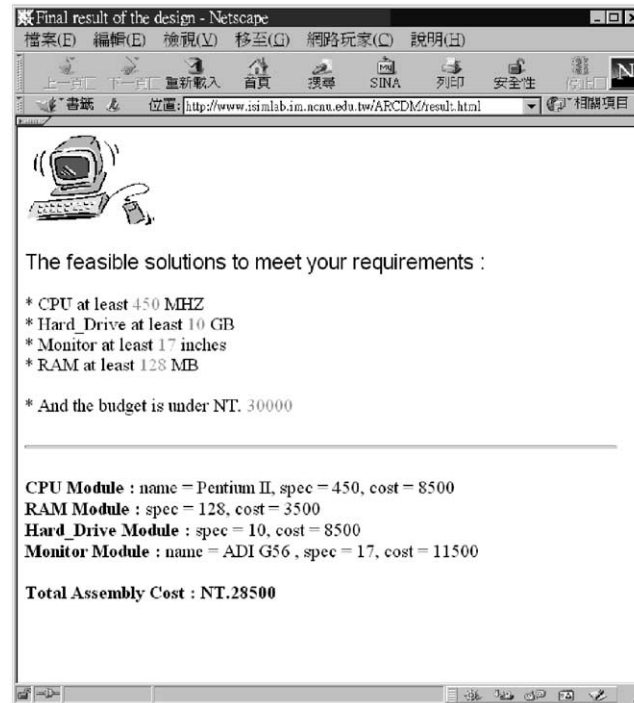


Fig. 7. Final result of the design.

structure consists of a with small number of modules assembled and a large number of alternatives available in each module, e.g., in this example, only four modules are needed to be assembled and each module includes large of alternatives, e.g., types of CPUs.

4. Conclusions

In this paper, the agent-based system of collaboration information and the procedure for designing with modules were presented. Since the remote collaborative development for modular product problems is quite complex, it is helpful if, at least, the solution to these problems is generated automatically by intelligent agents. This paper has attempted to generalize the design model, which included a wide variety of data types, constraints, and objectives that are considered. Such a system allows the designer to specify the problem's constraints flexibly, e.g., interface constraints and overall constraints, and then evolve the solutions step by step. One of the advantages of using this approach is that the designer can access the remote database all over the world instantaneously. Such corporations with the agent-based collaboration information character have great agility.

The approaches enhancing this product development system need to be explored in the future. One of them is to improve the searching procedure to make the system more effective and precise. Another possibility is to make this system respond to the designers' requirements immediately after each step. However, the practical application of the design for module

assembly through the collaboration information system has been proved. Doubtless, the future of the agent-based remote collaborative DwM application holds great promise and potential.

Acknowledgements

This work was partially supported by funding from the Nation Science Council of the Republic of China (NSC 89-2213-E-260-005, NSC 89-2213-E-212-008).

References

- Aoki, K. (1980). High speed and flexible automated assembly line—Why has automation successfully advanced in Japan? *Proceedings of the fourth international conference on production engineering* (pp. 1–6). Tokyo: Japan Society of Precision Engineering.
- Davidow, W., & Malone, M. (1992). *The virtual corporation*. New York: Harper Collins.
- Dewan, P., & Riedl, J. (1993). Toward computer-supported concurrent software engineering. *Computer*, 26(1), 12–16.
- Etzioni, O., & Weld, D. S. (1995). Intelligent agents on the Internet: Fact, fiction, and forecast. *IEEE Expert*, 10(4), 44–49.
- Fohn, S. M., Liao, J. S., Greef, A. R., Young, R. E., & O'Grady, P. J. (1995). Configuring computer systems through constraint-based modeling and interactive constraint satisfaction. *Computer in Industry*, 27(1), 3–21.
- Guilfoyle, C. (1995). Vendors of agent technology. *UNICOM Seminar on Intelligent Agents and their Business Application*, London, 8–9–November, pp. 135–142.
- Huang, C. C., & Liang, W. Y. (2000). The agent-based collaboration information system: case of modular product development. *Technical Report #0012*, Laboratory of Intelligent Systems and Knowledge Management, National Chi-Nan University, Taiwan.
- Jennings, N. R., Corera, J., Laresgoiti, I., Mamdani, E. H., Perriolat, F., Skarek, P., & Varga, L. Z. (1996). Using ARCHON to develop real-word DAI applications for electricity transportation management and particle accelerator control. *IEEE Expert*, 6(5), 64–70.
- Jennings, M. J., & Jennings, N. R. (1999). Software engineering with agent. *IEEE Internet Computing*, 3(3), 345–360.
- Jennings, N., & Wooldridge, M. (1996). Software agents. *IEE review*, 42(1), 17–20.
- Kidd, K. T. (1994). *Agile manufacturing: Forging new frontiers*. New York: Addison-Wesley.
- Kota, S., & Ward, A. C. (1990). Functions, structures, and constraints in conceptual design. In I. Rinderle (Ed.), *Proceedings of the second international conference on design theory and methodology* (pp. 239–250). Chicago, IL.
- Kusiak, A., & Huang, C. C. (1997). Design of modular digital circuits for testability. *IEEE Transactions on Components, Packaging, and Manufacturing Technology—Part C*, 20(1), 48–57.
- Kusiak, A., & Szczerbicki, E. (1992). A formal approach to specifications in conceptual design. *ASME Journal of Mechanical Design*, 114(12), 659–666.
- Morris, C. R., & Ferguson, C. F. (1993). How architecture wins technology wars. *Harvard Business Review*, 71(2), 86–96.
- Muller, J. P., Wooldridge, M. J., & Jennings, N. R. (1996). Proceedings of intelligent agent III: Agent theories, architectures, and languages. *ECAI '96 Workshop (ATAL)*, Budapest, Hungary.
- Nevins, J. L., & Whitney, D. E. (1989). *Concurrent design of products and processes: A strategy for the next generation in manufacturing*. New York: McGraw-Hill.
- Nwana, H. S. (1996). Software agents: An overview. *Knowledge Engineering Review*, 11(3), 205–244.
- Ovum (1994). Intelligent agents: The new revolution in software. *Ovum Report*.
- Pahl, G., & Beitz, W. (1988). *Engineering design*. New York: Springer-Verlag.
- Pine II, B. J. (1992). *Mass Customization: The new frontier in business competition*, Boston, MA: Harvard Business School Press.

- Sanchez, R. (1991). Strategic flexibility, real options, and product-based strategy, *Ph.D. dissertation*, Massachusetts Institute of Technology, Cambridge, MA.
- Sanchez, R. (1996). Strategic product creation: Managing new interactions of technology, markets, and organizations. *European Management Journal*, 14(2), 121–138.
- Sanchez, R., & Sudharshan, D. (1993). Real-time market research. *Marketing Intelligent and Planning*, 11(8), 29–38.
- Suh, N. P. (1990). *The principles of design*. Oxford, UK: Oxford University Press.
- Ulrich, K., & Tung, K. (1991). Fundamentals of product modularity. In A. Sharon, *Issues in design /manufacture integration 1991*, Vol. DE 39 (pp. 73–79). New York: ASME.
- Von Hippel, E. (1988). *The source of innovation*. New York: Oxford University Press.

Wen-Yau Liang is currently an assistant professor in the Department of Information Management of the Da-Yeh University. His research interests are remote collaborative design for modular products, distributed design, object-oriented design, product visualization, and electronic commerce. He received his Ph.D. degree in Industrial Engineering from the University of Iowa and the MBA degree in Industrial Management from the National Cheng-Kung University of Taiwan.

Chun-Che Huang is an associate professor in the Department of Information Management, National Chi-Nan University, Taiwan. He received his Ph.D. degree in Industrial Engineering from the University of Iowa, Iowa City, and the MS degree in Operations Research from Columbia University, New York, NY. He is interested in DSS, Intelligent Systems, Knowledge Management, and Concurrent Engineering.